TraceDesk · DEV-AI · Developer role

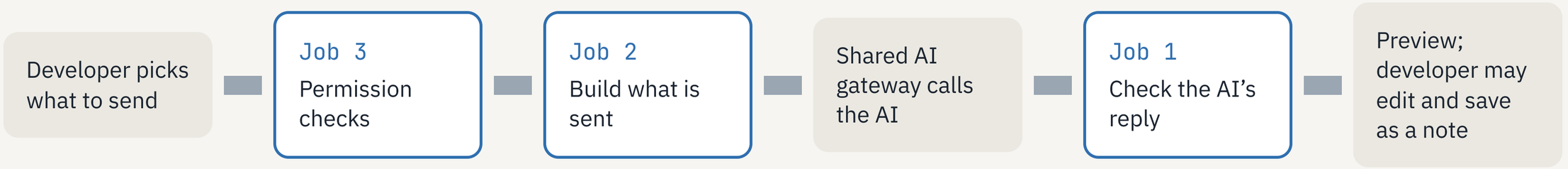
Less context, same code?

I gave two coding agents the same jobs. One could only read a short list of files, the other could read anything. A third agent then reviewed both without knowing which was which.

3 jobs · one pair each · GPT-6.1 Sol, medium effort · run 10–11 Nov 2026

The feature: AI suggestions for the developer

TraceDesk is a bug tracker. When a developer is working on a bug, they can ask the AI for possible causes and next steps. The AI only gives advice. It cannot change the bug report.



The three boxes in blue are the core I built in this experiment. Permission checks also run again before sending and before showing the result.

Spec: workflow/specs/developer-ai.md. Not built yet: the web endpoint, the screen, and saving as a note.

Two questions

Context · Lecture 9

If the agent reads less, is the code worse or just cheaper?

Checker · Lecture 8

Does a reviewer agent find enough to pay for itself?

I judged both by results: tests passed, bugs found, time taken. Not by how many tokens were used.

How I ran it

1 · Tests first

55 tests from the spec. Agents saw 41. I kept 14 outside the repo.

2 · Three jobs

Check the AI’s reply. Build what is sent. Permission checks.

3 · Two writers

A: 7 set docs. **B**: any file. Same time, separate folders.

4 · Blind check

A read-only agent reviews both, labelled X and Y.

Kept the same: model, effort, starting commit, and the prompt except for one line. Token counts come from Codex itself.

What A was allowed to read

File	What it is
`AGENTS.md`	Project rules
`spec-development skill`	How to build from a spec
`ai-integration skill`	How to work on AI code
`developer-ai.md`	The DEV-AI spec
`secured-ai-gateway.md`	Shared AI gateway spec
`ADR 002`	Shared role contracts
`ai-policy.md`	Rules for AI features

Plus, per job

That job’s starter class and test file, and any Java type needed to compile.

Not on the list

reporter-reports.md, which defines what “blank” text means. Job 1’s bug came from this gap.

The 7 docs are the set I picked by hand for DEV-AI in my earlier token measurement (9,848 tokens). Reused unchanged.

The 14 hidden tests

Job 1 · Reply

- Same key twice → reject
- 300 emoji count as 300, not 600
- Control characters → reject
- Broken emoji half → reject
- Spaces and accents kept exactly

Job 2 · What is sent

- Fields in spec order, not request order
- Notes with same time sorted by id
- Instruction under 1500 bytes
- 8000-byte limit counts bytes, not letters
- Edited note detected as out of date

Job 3 · Permissions

- Wrong developer refused before notes are read
- “Not allowed” beats “AI is off”
- Reassigned mid-request → not sent
- Second check re-reads the database

Kept in a folder outside the repo. Copied in only after a writer finished. Every version passed all 14.

Source: dev-ai-test-cases.md (cases V07 V11 V15 V17 V21, C02 C05 C09 C12 C13, A05 A11 A14 A16).

The prompts were the same except one line

Same for both

What to build · which test file defines it · “Follow the spec, not just the tests” · don’t edit tests · stay in this folder · don’t commit · the exact test command

A	`Context: read ONLY these files, and nothing else except Java types you must open to compile against: [the list]`
B	`Context: read whatever files in this repository you judge relevant.`
Rerun	`Same as A, plus: (read all of them together in one single command at the start)`

Both ran as: codex exec --json -m gpt-6.1-sol, effort medium, workspace-write sandbox.

Results: A and B passed the same tests

Job	A focused, input	B broad, input	What differed
1 · Check the reply	324K	400K	A missed one spec rule
2 · Build what is sent	338K	456K	Nothing, same quality
3 · Permission checks	1,510K	807K	A took 40 steps, B took 11
Total	**2,172K**	**1,662K**	**A used 31% more**

Both versions also passed all 14 hidden tests, so the tests could not tell them apart.

Input tokens per writer run, from Codex’s JSON usage report; 84–97% of input was cache reads. Source: runs.md, measurements.jsonl.

Job 1 · Check the AI’s reply

A missed a rule that was written in a file not on its list

A, focused

Counted a reply made only of non-breaking spaces as real text.

`"nextSteps": [" "] → accepted`

B, broad

Found the Reporter spec, where “blank” is defined, and the existing code for it.

`"nextSteps": [" "] → rejected`

None of the 55 tests checked this. A was 19% cheaper but had one spec bug.

Our docs’ fault too: “blank” is defined inside each role’s spec, never in one shared place, and the DEV-AI spec left it out.

Job 3 · Permission checks, plus one rerun

More steps, more tokens: reading one file at a time was half of A’s extra cost

1.51M

A: list, one file per step. 40 steps.

1.13M

A rerun: same list, read in one step. 12 steps.

0.81M

B: chose its own files. 11 steps.

Each step sends the whole conversation again. All three passed the same tests. The rerun still cost 40% more than B, because it carried the whole list in every step.

The checker

The checker found 2 problems the tests missed

0.68M

input tokens for all 3 reviews together

⅓

of a writer run’s tokens, per review on average

2

real problems: the blank-text bug, and a wrong error code

I think it was worth it here: a small extra cost for problems no test caught.

How careful to be with these results

Only 3 pairs, run once each

Another run could come out differently. This is what I saw, not a rule.

The checker is the same model

It may share the writers' blind spots. I checked every finding by hand.

I picked A's file list

The rule A missed sat in the wrong doc. With better docs, A might have got job 1 right.

Times are rough

A and B ran together on my laptop and shared its CPU. Token counts are not affected.

What I will do differently

Change	Stop giving a fixed reading list by default. Left free, the agent read 17–28 files, not all 98 linked docs.
Add	Ask agents to read in batches and run tests once. Steps cost tokens, files mostly don’t.
Keep	A blind, read-only checker for work with a detailed spec.
Add	Tests for the two problems the checker found, so the tests catch them next time.